

Deployment (Netlify/Vercel)

Ship Your Project to the World

■ LEARNING OUTCOME

By the end of this module you will deploy a full-stack application to production with a custom domain, HTTPS, environment variables, automated CI/CD, and proper monitoring.

■ Skills You Will Gain

- Deploy a React frontend to Vercel with automatic CI/CD
- Deploy a Node.js backend to Railway or Render
- Configure environment variables for production
- Set up a custom domain with SSL/HTTPS
- Configure GitHub Actions for automated testing and deployment
- Monitor uptime and errors in production

■ **Deployed = Real.** An app running on your laptop is a prototype. A deployed app with a URL proves it works in the real world. Deploy early (even day 1) and deploy often -- catch production issues before launch.

SECTION 1

Frontend Deployment to Vercel

1	Push to GitHub Create public GitHub repo. git push all code. Ensure .gitignore excludes node_modules and .env files.
2	Connect to Vercel Sign up at vercel.com with GitHub. Click New Project. Select your repository. Auto-detects Vite/React.
3	Configure Build Build Command: npm run build. Output Directory: dist. Framework Preset: Vite. Auto-detected usually.
4	Add Environment Variables Settings > Environment Variables. Add VITE_API_URL pointing to your backend URL. Separate values for preview vs production.
5	Deploy Click Deploy. Vercel builds and deploys in ~60 seconds. Get: username.vercel.app URL immediately.
6	Custom Domain Settings > Domains. Add yourdomain.com. Vercel gives CNAME/A records. Add in your domain registrar DNS. Auto SSL in minutes.
7	Auto Deploy Every git push to main automatically redeploys production. Every PR gets a unique preview URL for review.

SECTION 2

Backend Deployment to Railway

```
# RAILWAY DEPLOYMENT (free tier: $5 credit/month) # railway.app -- connects to GitHub, deploys
Node.js automatically # 1. Add start script to package.json { 'scripts': { 'start': 'node
server.js', 'dev': 'nodemon server.js'}, 'engines': { 'node': '>=18.0.0' } } # 2. Procfile
(optional but explicit) web: node server.js # 3. Use PORT from environment (Railway sets this)
const PORT = process.env.PORT || 3000; app.listen(PORT, () => console.log(`Server running on port
${PORT}`)); # 4. Railway environment variables # Go to Railway dashboard > your service > Variables
# Add ALL your .env variables: NODE_ENV=production DATABASE_URL=postgresql://... (Railway can
provision PostgreSQL too) JWT_SECRET=long-random-secure-string-min-32-chars
JWT_REFRESH_SECRET=different-long-random-string CLIENT_URL=https://yourapp.vercel.app
GOOGLE_CLIENT_ID=... GOOGLE_CLIENT_SECRET=... # 5. Database on Railway # Dashboard > New > Database
> PostgreSQL # Railway provides DATABASE_URL automatically as env var # Connect Prisma:
DATABASE_URL is already set npx prisma migrate deploy # run migrations on production database #
RENDER ALTERNATIVE (free tier available) # render.com -- similar to Railway # Add: render.yaml to
root services: - type: web name: my-api env: node buildCommand: npm install && npx prisma migrate
deploy startCommand: node server.js envVars: - key: NODE_ENV value: production
```

SECTION 3

GitHub Actions CI/CD Pipeline

```
# .github/workflows/deploy.yml name: CI/CD Pipeline on: push: branches: [main] pull_request:
branches: [main] jobs: # JOB 1: Test everything test: name: Test runs-on: ubuntu-latest services:
postgres: image: postgres:15 env: POSTGRES_USER: testuser POSTGRES_PASSWORD: testpass POSTGRES_DB:
testdb options: >- --health-cmd pg_isready --health-interval 10s steps: - uses: actions/checkout@v4
- uses: actions/setup-node@v4 with: { node-version: '20', cache: 'npm' } - name: Install and test
backend working-directory: ./backend run: | npm ci npx prisma migrate deploy npm test env:
DATABASE_URL: postgresql://testuser:testpass@localhost:5432/testdb JWT_SECRET:
test-secret-at-least-32-characters - name: Install and test frontend working-directory: ./frontend
run: | npm ci npm test -- --run npm run build env: VITE_API_URL: http://localhost:3000 # JOB 2:
Deploy (only on main, only if tests pass) deploy-backend: name: Deploy Backend needs: test runs-on:
ubuntu-latest if: github.ref == 'refs/heads/main' steps: - uses: actions/checkout@v4 - name: Deploy
to Railway uses: bervProject/railway-deploy@main with: railway-token: ${ secrets.RAILWAY_TOKEN }}
service: ${ secrets.RAILWAY_SERVICE_ID }} deploy-frontend: name: Deploy Frontend needs: test
runs-on: ubuntu-latest if: github.ref == 'refs/heads/main' steps: - uses: actions/checkout@v4 -
name: Deploy to Vercel uses: amondnet/vercel-action@v25 with: vercel-token: ${ secrets.VERCEL_TOKEN }}
vercel-org-id: ${ secrets.VERCEL_ORG_ID }} vercel-project-id: ${ secrets.VERCEL_PROJECT_ID }}
vercel-args: '--prod'
```

SECTION 4

Production Checklist

Category	Checklist Item
Security	HTTPS enforced, HTTP redirects to HTTPS, Content-Security-Policy header set, rate limiting active
Environment	NODE_ENV=production, all .env vars set, no dev dependencies in prod, no debug logging
Database	Migrations run, connection pooling configured, backups enabled, no development data
Performance	Static assets cached (max-age), images optimised, gzip/brotli compression enabled
Monitoring	Uptime monitoring (UptimeRobot free), error tracking (Sentry free tier), logging enabled
Domain	Custom domain pointing to app, SSL certificate valid, www redirects to apex or vice versa
Testing	All critical user flows tested manually on production, mobile tested, slow connection tested

■ PRACTICE Q & A

Q1. What is the difference between a preview deployment and a production deployment?

A: *Preview: every PR gets a unique URL (my-app-abc123.vercel.app) for review and testing. Production: merging to main deploys to your main URL (myapp.com). Preview deployments use preview env vars; production uses production env vars.*

Q2. Why should you set NODE_ENV=production in deployment?

A: *Disables development-only features: stack traces in error responses, Morgan verbose logging, Prisma query logging. Enables production optimisations: Express caches view templates, certain middleware enables production-only behaviour.*

Q3. What is the difference between npm install and npm ci?

A: *npm install: updates package-lock.json, installs dependencies. npm ci: installs EXACTLY from package-lock.json, deletes node_modules first. npm ci is deterministic and faster in CI -- ensures everyone uses the exact same versions.*

Q4. How do you store sensitive environment variables for CI/CD?

A: *GitHub Actions: store in repository Settings > Secrets > Actions. Reference as `${{ secrets.MY_SECRET }}`. Secrets are encrypted, never shown in logs, and only available to workflows. Never hardcode secrets in workflow YAML files.*

Q5. What is a health check endpoint and why do you need one?

A: *GET /health returns `{status:'ok',timestamp:'...'}` with 200. Load balancers, Railway, Render, and uptime monitors use this to check if your app is running. Without it, they cannot distinguish between 'app is up' and 'app crashed'.*

■ Deployment Cheat Sheet	
<code>npm run build</code>	Build React app for production
<code>process.env.PORT 3000</code>	Use platform's port
<code>NODE_ENV=production</code>	Enable production mode
<code>npm ci</code>	Deterministic install from lock file
<code>npx prisma migrate deploy</code>	Run DB migrations in prod
<code>engines: { node: '>=18' }</code>	Specify Node version
<code>GET /health -> {status:'ok'}</code>	Health check endpoint
<code>secrets.MY_SECRET</code>	GitHub Actions secure secrets
<code>on: push branches: [main]</code>	Deploy on main push
<code>needs: test</code>	Deploy only if tests pass
<code>--prod flag (Vercel)</code>	Deploy to production (not preview)
<code>UptimeRobot free</code>	Monitor uptime every 5 minutes